Leakage-Aware Preprocessing and Multi-Model Validation for the Spaceship Titanic Classification Task

Wang Han 2430034066 Shen Yijie 2430034056 Yang Yusong 2430034087
Jiang Zhibo 2430034031 He Yuxiang 2230024062
group3

***Abstract*—This paper studies the Kaggle Spaceship Titanic binary classification task using a leakage-aware tabular machine-learning workflow. The proposed pipeline combines deterministic cleaning rules, train-only preprocessing statistics, engineered demographic and spending features, and branch-specific representations for Logistic Regression, Random Forest, XGBoost, LightGBM, and CatBoost. Under a stratified 80/20 local holdout protocol, LightGBM records the highest local validation accuracy (0.8171), while CatBoost records the strongest ROC-AUC (0.9125). For external submission feedback, the final XGBoost submission reaches a Kaggle public leaderboard score of 0.81412. These results suggest that careful preprocessing and representation design contribute as much to final performance as lightweight hyperparameter tuning in this tabular setting.**

***Index Terms*—Spaceship Titanic, tabular machine learning, feature engineering, logistic regression, random forest, XGBoost, LightGBM, CatBoost**

I. Introduction

The Spaceship Titanic competition is a binary classification task in which the goal is to predict whether a passenger was transported to an alternate dimension based on demographic, cabin, travel-destination, and spending information [1]. As a representative tabular classification problem, it sits in a regime where strong tree-based methods, careful feature representation, and disciplined preprocessing often remain highly competitive relative to heavier deep tabular pipelines [2]–[6]. The study is organized around two layers:

- a minimal runtime layer for preprocessing, full training, and inference, and
- an independent local validation layer for controlled model comparison, report-ready figures, and lightweight tuning.

The aim of this report is to document these components precisely: the preprocessing rules are written down explicitly, the model structures are formalized with equations and diagrams, and the parameter settings are reported in exact table form. This emphasis on explicit split handling, reproducible preprocessing, and transparent evaluation follows recent recommendations on leakage-aware machine-learning reporting [7], [8].

II. Task and Dataset

A. Prediction Task

Let x_i denote the feature vector for passenger i and let $y_i \in \{0, 1\}$ denote the label, where $y_i = 1$ corresponds to Transported=True. The task is to learn a mapping

$$\hat{y}_i = f(x_i) \tag{1}$$

or, more generally, a probabilistic score

$$\hat{p}_i = P(y_i = 1 \mid x_i), \tag{2}$$

which can later be converted to a binary decision with a threshold τ:

$$\hat{y}_i = \begin{cases} 1, & \hat{p}_i \geq \tau, \\ 0, & \hat{p}_i < \tau. \end{cases} \tag{3}$$

B. Dataset Summary

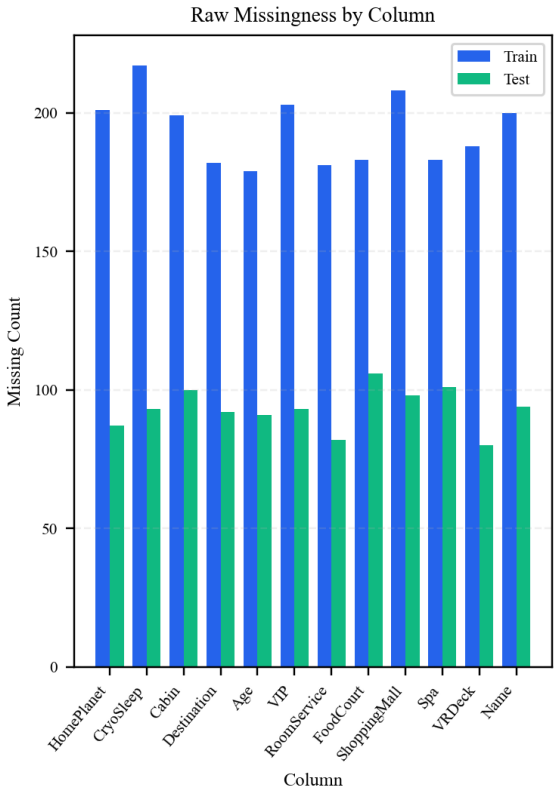
This study uses the official Kaggle *Spaceship Titanic* splits [1]:

- train.csv: 8693 labeled rows,
- test.csv: 4277 unlabeled rows.

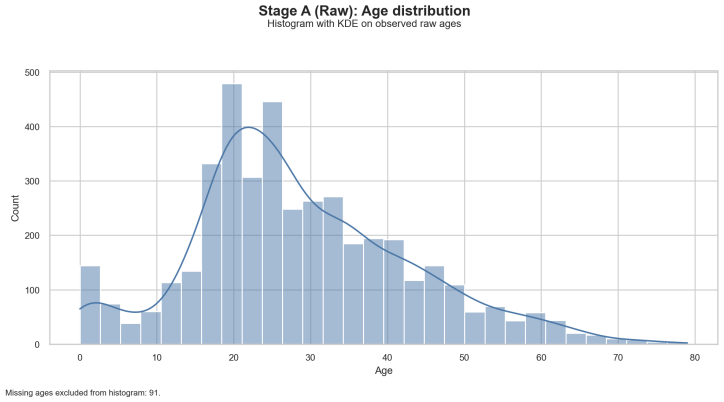
The raw training file contains the following columns: PassengerId, HomePlanet, CryoSleep, Cabin, Destination, Age, VIP, RoomService, FoodCourt, ShoppingMall, Spa, VRDeck, Name, and the binary target Transported.

TABLE I
Dataset summary

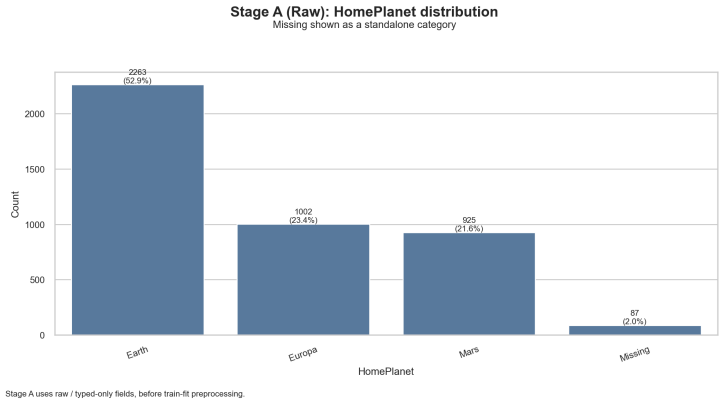
Item	Value	Notes
Training rows	8693	labeled
Test rows	4277	unlabeled Kaggle split
Positive-class ratio	0.5036	train only
Raw training missing values	2324	before cleaning
Raw test missing values	1117	before cleaning
Target column	Transported	binary label



(a) Raw missing-value counts across train and test files.



(b) Observed raw age distribution in the training data.



(c) Raw HomePlanet distribution with missing values retained as a separate category.

Fig. 1. Initial exploratory data analysis for the raw Spaceship Titanic dataset. The missingness plot compares raw train and test missing-value counts, while the age and HomePlanet plots summarize key demographic distributions before preprocessing.

III. Shared Preprocessing Pipeline

A. Overview

The shared preprocessing logic is implemented in preprocess.py. Its purpose is to transform raw Kaggle records into a stable common feature frame and then branch that frame into model-specific representations. The pipeline first normalizes raw fields, then computes train-fit statistics, then creates shared engineered variables, and finally emits branch-specific bundles for each model family. This design treats cleaning and feature engineering as first-class parts of tabular model development rather than as a disposable preamble [6], [9].

B. Raw Cleaning and Type Normalization

The first stage applies two deterministic operations:

1) **Basic cleaning**: whitespace-only strings are converted to missing values and string fields are stripped of surrounding spaces.
2) **Type enforcement**: numeric columns such as Age and the five spending variables are coerced to numeric values; CryoSleep and VIP are normalized into string-valued boolean-like tokens (True/False); Transported is cast to integer 0/1.

These operations follow standard recommendations for robust data cleaning and schema normalization before downstream modeling [9].

C. Identifier Parsing

Two important structural columns are extracted directly from the raw identifiers:

$$\text{PassengerId} = \text{GroupID\_GroupMemberNo}, \tag{4}$$

$$\text{Cabin} = \text{Deck/CabinNum/Side}. \tag{5}$$

Concretely:

- GroupID is the substring before the underscore in PassengerId.
- GroupMemberNo is the numeric suffix after the underscore.
- Deck, CabinNum, and Side are obtained by splitting Cabin on the slash delimiter.
- Surname is extracted as the last token in Name.

D. Group-based Deterministic Rules

The preprocessing stage also applies rule-based filling before statistical imputation:

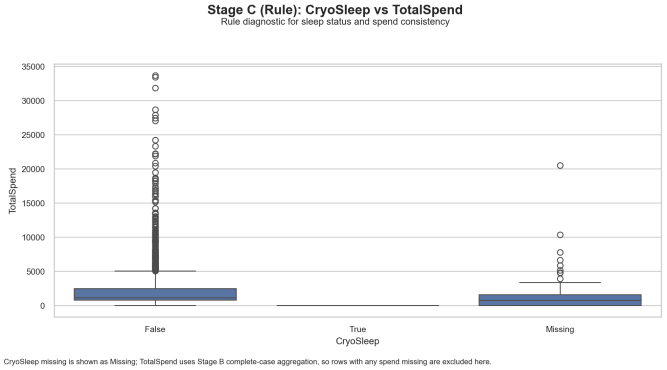


Fig. 2. Relationship between CryoSleep status and TotalSpend, supporting the zero-spend consistency rule used during preprocessing.

- HomePlanet, Destination, and VIP can be filled from GroupID when all observed values inside the group are consistent.
- Missing CryoSleep is inferred as False when any spend column is strictly positive.
- Missing CryoSleep is inferred as True when the passenger is an adult, all five spend columns are observed, and total spend equals zero.
- For rows with CryoSleep=True, all spend columns are deterministically forced to zero.

Such rules intentionally exploit structured consistency in the raw table before broader imputation, while keeping the logic explicit enough to audit during validation and later error analysis [10], [11].

E. Train-fit Statistics

The pipeline explicitly separates train-fit statistics from later transformations, because reusing full-split statistics during validation would introduce leakage and weaken reproducibility claims [7], [8]. The following quantities are fit on training data only:

- age median after out-of-range filtering,
- SurnameFreq lookup map,
- global spend medians,
- positive-spend medians,
- hierarchical spend medians,
- CabinNumBin quantile edges.

Age normalization is defined as

$$\tilde{a}_i = \begin{cases} a_i, & 0 \le a_i \le 100, \\ \mathrm{NaN}, & \text{otherwise}, \end{cases} \tag{6}$$

and an indicator

$$\mathrm{AgeWasOutOfRange}_i = \mathbb{1}[a_i < 0 \ \vee \ a_i > 100] \tag{7}$$

is retained as a feature.

TABLE II
TRAIN-FIT STATISTICS AND DETERMINISTIC RULES

Component	Implementation detail
Age normalization	Values outside $[0, 100]$ are flagged and treated as missing before median fill.
Surname frequency	SurnameFreq is computed from training surnames only; unseen test surnames map to 0.
Cabin binning	CabinNum is binned with train-fit quantile edges; missing values go to CabinBin_Missing.
Spend imputation	Group median → positive median → global median, after enforcing the CryoSleep zero-spend rule.
Group size	Full preprocessing uses combined train+test passenger groups; local validation recomputes group size split-locally.

For spending variables, the imputation hierarchy is:

$$x_{ij}^{\mathrm{filled}} = \begin{cases} 0, & \mathrm{CryoSleep}_i = \mathrm{True}, \\ \text{group median}, & \text{if a matching hierarchy bucket exists}, \\ \text{positive median}_j, & \text{if other spend categories indicate positive sp} \\ \text{global median}_j, & \text{otherwise}. \end{cases} \tag{8}$$

The hierarchy buckets used for group medians are:

- HomePlanet|Destination|VIP|Deck,
- HomePlanet|Destination|Deck,
- HomePlanet|Deck.

This hierarchy favors transparent, split-local heuristics over heavier iterative imputers; stronger alternatives such as automatic model-selection-based imputation remain possible future upgrades rather than hidden preprocessing shortcuts [10]–[12].

F. Shared Engineered Features

The project creates a large engineered feature set rather than relying on raw variables alone. Several of the most important features are defined explicitly below. This choice is consistent with recent evidence that feature design and data representation remain major determinants of tabular-model quality even when strong model families are already available [5], [6], [9].

1) Spend Features: Let the spend columns be

$$\mathcal{S} = \{\mathrm{RoomService}, \mathrm{FoodCourt}, \mathrm{ShoppingMall}, \mathrm{Spa}, \mathrm{VRDeck}\}.$$

Then

$$\mathrm{TotalSpend}_i = \sum_{s \in \mathcal{S}} s_i, \tag{9}$$

$$\mathrm{SpendCount}_i = \sum_{s \in \mathcal{S}} \mathbb{1}[s_i > 0], \tag{10}$$

$$\mathrm{IsZeroSpend}_i = \mathbb{1}[\mathrm{TotalSpend}_i = 0]. \tag{11}$$

Luxury/basic decompositions are defined as

$$\mathrm{LuxurySpend}_i = \mathrm{Spa}_i + \mathrm{VRDeck}_i, \tag{12}$$

$$\mathrm{BasicSpend}_i = \mathrm{RoomService}_i + \mathrm{FoodCourt}_i + \mathrm{ShoppingMall}_i, \tag{13}$$

$$\mathrm{LuxuryShare}_i = \frac{\mathrm{LuxurySpend}_i}{\mathrm{TotalSpend}_i + 1}, \tag{14}$$

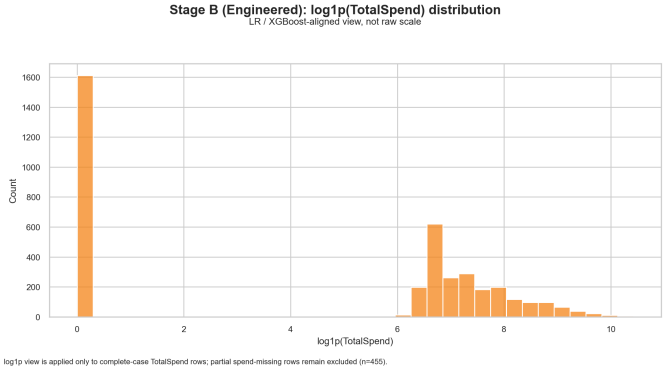


Fig. 3. Distribution of log-transformed TotalSpend used to reduce the skewness of spending-related features.

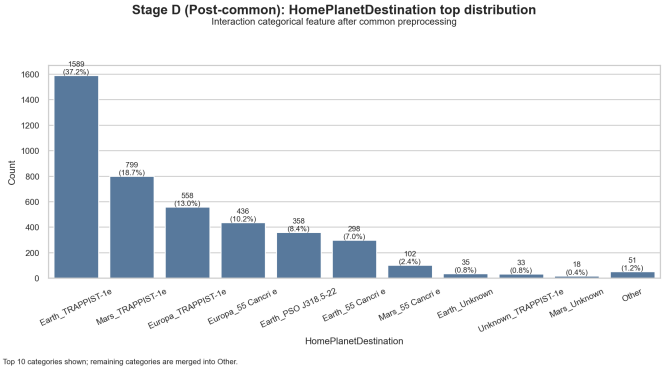


Fig. 4. Top HomePlanetDestination interaction categories after common preprocessing.

$$\text{SpendPerActiveCategory}_i = \frac{\text{TotalSpend}_i}{\max(\text{SpendCount}_i, 1)}. \quad (15)$$

2) Age and Group Features: Age segmentation is encoded through

$$\text{IsChild}_i = \mathbb{1}[\text{Age}_i < 18], \quad (16)$$

$$\text{IsSenior}_i = \mathbb{1}[\text{Age}_i \geq 60], \quad (17)$$

and AgeGroup is binned into the semantic intervals: Child, Teen, YoungAdult, Adult, MiddleAge, Senior, and Unknown.

Group features include

$$\text{IsSolo}_i = \mathbb{1}[\text{GroupSize}_i = 1], \quad (18)$$

$$\text{IsMultiPassengerGroup}_i = \mathbb{1}[\text{GroupSize}_i > 1], \quad (19)$$

$$\text{GroupMemberIsLeader}_i = \mathbb{1}[\text{GroupMemberNo}_i = 1]. \quad (20)$$

3) CatBoost-specific Extra Features: For the CatBoost branch, extra features are added beyond the common base features:

$$\text{GroupMemberRatio}_i = \frac{\text{GroupMemberNo}_i}{\max(\text{GroupSize}_i, 1)}, \quad (21)$$

$$\text{CabinNumLog1p}_i = \log(1 + \max(\text{CabinNum}_i, 0)), \quad (22)$$

$$\text{SpendNonZeroRatio}_i = \frac{\text{SpendCount}_i}{5}, \quad (23)$$

$$\text{LuxuryToBasicRatio}_i = \frac{\text{LuxurySpend}_i}{\text{BasicSpend}_i + 1}, \quad (24)$$

$$\text{SpendPerGroupMember}_i = \frac{\text{TotalSpend}_i}{\max(\text{GroupSize}_i, 1)}. \quad (25)$$

Additional categorical interaction features are DeckCabinBin, DeckDestination, and SurnameFreqBin. Retaining and combining categorical signals in this way is consistent with recent findings on encoder choice and interpretable learning over large categorical domains [13], [14].

The shared feature inventory can be summarized in five blocks: structural identifiers (PassengerId, GroupID, GroupMemberNo, GroupSize, Deck, CabinNum, Side, Surname); common categorical fields (HomePlanet, CryoSleep, Destination, VIP, Deck, Side, AgeGroup, CabinNumBin, DeckSide, HomePlanetDestination); base numeric features (age, five spending columns, total-spend aggregates, group statistics, and binary flags); explicit missingness indicators for the main raw columns; and CatBoost-only extra numeric/categorical features such as GroupMemberRatio, CabinNumLog1p, SpendStd, DeckCabinBin, and SurnameFreqBin. This inventory deliberately treats missingness indicators and cross-field interactions as model inputs rather than as mere cleaning artifacts, which is consistent with recent analyses of supervised learning with incomplete tabular data [10].

G. Branch-specific Representations

After the common frame is finalized, each model receives a dedicated representation:

- Logistic Regression: log1p on spend-related features, numeric standardization, and one-hot encoded categorical variables.
- Random Forest: raw numeric passthrough with one-hot encoded categoricals and no scaling.
- XGBoost: log1p spend transforms plus one-hot encoded categoricals, but no scaling.
- LightGBM: native pandas category columns with aligned train/test levels.
- CatBoost: native string categorical columns, including Surname, plus branch-specific extra engineered features.

This branch-specific encoding choice is motivated by recent encoder benchmarks and tabular-model comparisons, which show that representation choice can materially change downstream ranking even when the learner family is held fixed [3], [4], [13].

This branch split is the exact place where the five models stop sharing a single feature representation.

In implementation terms, the five branches differ as follows. Logistic Regression applies log1p to spend-related columns, standardizes the numeric block, one-hot encodes the categoricals, and fits a sparse 107-feature linear classifier. Random Forest keeps the engineered numeric values on their original scale, one-hot encodes the same categorical block, and fits a sparse 107-feature tree ensemble. XGBoost also uses a 107-feature encoded matrix, but combines log1p numeric transforms with

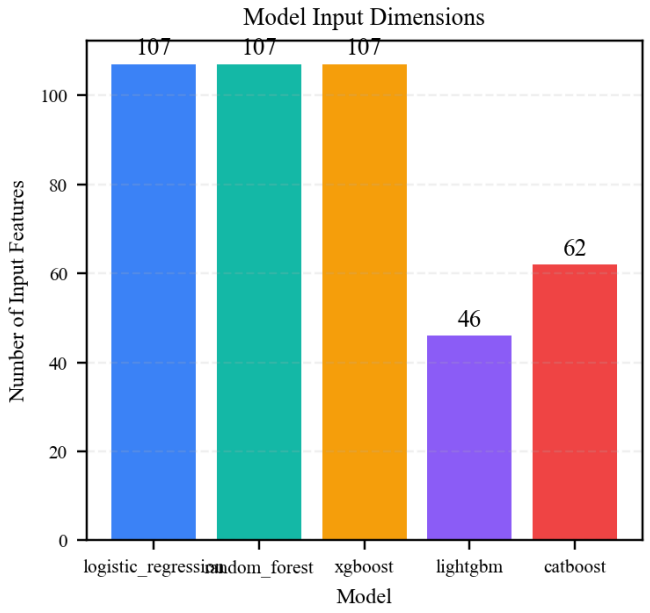


Fig. 5. Dimensionality of the baseline local-validation model inputs.

tree-based boosting rather than scaling. This 107-feature XGBoost representation is the controlled local-validation baseline, not the final Kaggle-facing recovery submission. LightGBM keeps a 46-column DataFrame with aligned pandas categorical levels. CatBoost keeps a 62-column mixed-type DataFrame with extra engineered features and explicit categorical indices.

IV. Model Structures and Parameterization

A. Unified Structural View

Although all five models solve the same binary classification problem, their internal structure is different. The project includes one linear probabilistic classifier, one bagged tree ensemble, and three gradient-boosted tree branches with different categorical-handling strategies. This mix is representative of modern strong baselines for heterogeneous tabular data [2], [5].

B. Logistic Regression

The Logistic Regression branch uses a linear score and sigmoid probability:

$$z_i = w^\top x_i + b, \tag{26}$$

$$\hat{p}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}. \tag{27}$$

The current validation baseline is implemented with scikit-learn [15] and uses solver=lbfgs, C=1.0, max_iter=1000, and random_state=42. In the lightweight search, the tuned best configuration was C=0.01, penalty=l2, solver=lbfgs, max_iter=1000. This branch serves as the clearest linear baseline in the repository. Recent work shows that logistic-style models can remain useful when paired with interpretable feature engineering, while their interpretability still depends on feature coding, scaling, and user context rather than following automatically from the model name alone [16], [17].

C. Random Forest

Random Forest builds an ensemble of decision trees on bootstrapped samples [18]. Each tree is trained on a randomly sampled subset of the training data, and different trees may learn different decision patterns. This design helps reduce the dependence on a single decision tree and improves the stability of the final prediction.

The output probability can be written as

$$\hat{p}_i = \frac{1}{T} \sum_{t=1}^{T} p_t(x_i), \tag{28}$$

where T is the number of trees, and $p_t(x_i)$ denotes the predicted probability produced by the t-th decision tree for sample x_i. The final probability $\hat{p}_i$ is obtained by averaging the probability outputs of all trees. Therefore, the final prediction is based on the collective decision of the whole forest rather than one individual tree.

The current validation baseline uses n_estimators=400, max_depth=None, min_samples_split=2, min_samples_leaf=1, and max_features=sqrt. Here, n_estimators=400 means that the model builds 400 decision trees. max_depth=None allows each tree to grow until the stopping conditions are reached. min_samples_split=2 means that a node can be split when it contains at least two samples, while min_samples_leaf=1 allows each leaf node to contain at least one sample. The parameter max_features=sqrt means that only a random subset of features is considered at each split, which increases diversity among trees and helps reduce overfitting [18].

Unlike Logistic Regression, Random Forest does not require feature scaling because decision trees split data according to feature thresholds rather than distance-based or gradient-based calculations. Therefore, the Random Forest branch keeps numerical features on their original scale and applies one-hot encoding to categorical variables. This remains consistent with recent tabular-learning comparisons that treat Random Forest as a strong out-of-the-box baseline, while noting that interpretation usually shifts from coefficients to importance-style summaries and variable-selection analysis [3], [19].

D. XGBoost

XGBoost is implemented as additive gradient-boosted trees [20]:

$$F_M(x) = \sum_{m=1}^{M} \eta f_m(x), \tag{29}$$

where η is the learning rate and each f_m is a regression tree. The training objective at boosting step t is

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t). \tag{30}$$

The current baseline uses n_estimators=500, max_depth=4, learning_rate=0.05, subsample=0.9, colsample_bytree=0.9,

TABLE III
LightGBM top-feature configuration

Setting	Value	Setting	Value
Objective	binary	L1 reg.	0.2
Boosting type	gbdt	L2 reg.	2.5
Maximum budget	5000 trees	max_depth	-1
Learning rate	0.03	Min split gain	0.02
Num. leaves	23	Subsample freq.	1
Min child samples	40	Final refit trees	350
Row subsample	0.85	Selected features	30
Column subsample	0.75	Best public LB	0.80547

and tree_method=hist. This baseline configuration corresponds to the local validation comparison in Table VI.

Beyond this controlled validation baseline, XGBoost was maintained as a separate submission-oriented experimental branch. This branch used StratifiedGroupKFold to reduce passenger-group leakage, recomputed fold-dependent statistics such as SurnameFreq and CabinNumBin inside validation folds, and applied Optuna tuning after the validation design was leakage-aware. Earlier target-encoding variants produced inflated OOF scores and weaker public leaderboard behavior, so unsafe target-encoding features were removed from the clean XGBoost line. The final submitted file, `submission_team_common_rawlike_feedback_rank_k3_seed17_w05_true2291.csv`, was selected from documented submitted candidates and reached a Kaggle public leaderboard score of 0.81412. This score is reported as public submission evidence, not as a private-leaderboard or generalization guarantee [8].

E. LightGBM

LightGBM follows the same additive boosting template [21],

$$F_t(x) = F_{t-1}(x) + \eta f_t(x), \tag{31}$$

but the current implementation exploits two practical details:

- branch-specific native categorical columns are preserved as pandas category dtype,
- train and test category levels are aligned before fitting.

The analyzed LightGBM top-feature branch uses the configuration summarized in Table III. In the selected feature-ablation run, gain importance retained 30 features and the final refit used n_estimators=350. This 30-feature setting is chosen because it has the best recorded Kaggle public leaderboard score, 0.80547, within the analyzed top-feature LightGBM branch, which is consistent with recent benchmark evidence that strong boosted-tree defaults remain highly competitive on tabular tasks [5], [22]. Native handling of aligned categorical columns is one reason this branch remains competitive on heterogeneous tabular data and in data-centric benchmark settings [3], [6], [13].

F. CatBoost

CatBoost is also an additive tree model [23], but it is particularly suited to mixed numeric and categorical tabular inputs. The implementation in this project keeps categorical columns as strings and passes the column indices explicitly to the learner. The decision to retain Surname and related categorical interactions in this branch follows recent work showing that high-cardinality categorical fields can be useful when the model family is designed to absorb them directly [13], [14]. The baseline uses iterations=500, depth=6, learning_rate=0.05, loss_function=Logloss, eval_metric=Accuracy, and random_seed=42.

V. Optimization and Validation Design

A. Project-side Training Scripts

The repository contains both the lightweight validation layer and standalone model-training scripts. The validation layer is designed for controlled comparison; the training scripts are closer to the actual submission workflow. Two operations recur across these scripts:

$$\hat{y}_i(\tau) = \mathbb{1}[\hat{p}_i \geq \tau], \tag{32}$$

which defines thresholded classification, and, for multi-seed ensembles,

$$\bar{p}_i = \frac{1}{S} \sum_{s=1}^{S} \hat{p}_i^{(s)}, \tag{33}$$

which averages predicted probabilities over S random seeds before thresholding. Choosing τ on out-of-fold or held-out probabilities is consistent with recent threshold-optimization and calibration guidance, provided the threshold is selected inside the validation loop rather than on the final test split [7], [24]–[26].

The project-side training scripts are more specialized than the lightweight validation layer. Logistic Regression fits the frozen sparse LR bundle with solver=saga, penalty=l2, C=1.0, and fixed threshold $\tau = 0.5$, whereas Random Forest uses a tuned high-capacity recipe (n_estimators=2600, max_depth=16, criterion=log_loss, class_weight=balanced) and stores an accuracy-optimized OOF threshold. The XGBoost submission branch is handled separately from the lightweight validation layer because it has its own group-aware CV, fold-aware preprocessing, Optuna search, ablation, leak diagnosis, SHAP/importance, multi-seed bagging, and final submission-selection workflow. LightGBM uses 5-fold StratifiedKFold CV, 250-round early stopping on validation binary_logloss, OOF-based threshold tuning, and a final refit with the selected 30-feature subset and n_estimators=350. CatBoost uses iterations=2000, learning_rate=0.03, depth=6, l2_leaf_reg=5.0, and 5-fold CV with OOF threshold search in its cv_tune mode.

TABLE IV
EXACT BASELINE PARAMETERS AND CURRENT LIGHTGBM SETTING

Model	Exact configuration in the current validation layer
Logistic Regression	max_iter=1000, solver=lbfgs, C=1.0, random_state=42.
Random Forest	n_estimators=400, max_depth=None, min_samples_split=2, min_samples_leaf=1, max_features=sqrt, n_jobs=-1, random_state=42.
XGBoost	n_estimators=500, max_depth=4, learning_rate=0.05, subsample=0.9, colsample_bytree=0.9, eval_metric=logloss, tree_method=hist, n_jobs=-1, random_state=42.
LightGBM	objective=binary, boosting_type=gbdt, n_estimators=5000, learning_rate=0.03, num_leaves=23, min_child_samples=40, subsample=0.85, colsample_bytree=0.75, reg_alpha=0.2, reg_lambda=2.5, max_depth=-1, min_split_gain=0.02, subsample_freq=1, n_jobs=-1.
CatBoost	iterations=500, depth=6, learning_rate=0.05, loss_function=Logloss, eval_metric=Accuracy, verbose=False, allow_writing_files=False, random_seed=42.

B. Holdout Validation Protocol

The current comparison uses a stratified holdout split:

$$(\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{valid}}) = \text{Split}_{0.8/0.2}(\mathcal{D}; \text{random\_state} = 42, \text{stratify} = \text{Transported}). \tag{34}$$

This produces 6954 local training rows and 1739 local validation rows. All train-fit preprocessing statistics are then recomputed on the local training partition only, which is essential for leakage control and for making the resulting model comparison reproducible [7], [8].

C. Metrics

The validation runner records accuracy, precision, recall, F1, ROC-AUC, fit_seconds, and predict_seconds. Accuracy is the primary ranking metric, but it is not the only one retained because threshold quality, ranking quality, and probability calibration answer different evaluation questions [24], [25]. This is why the project also keeps ROC curves, probability distributions, calibration plots, and threshold sweeps rather than reporting a single thresholded score [26].

D. Hyperparameter Search Spaces

The validation layer implements a lightweight tuning module. The current lightweight tuning table covers Logistic Regression, Random Forest, LightGBM, and CatBoost. XGBoost has a baseline validation configuration in Table IV, while its stronger local experiments are tracked in the separate XGBoost module rather than in this lightweight cross-model tuning snapshot.

The lightweight validation search spaces are deliberately small. Logistic Regression tunes only C over $\{0.01, 0.1, 1.0, 3.0, 10.0\}$. Random Forest varies tree count, depth, split thresholds, leaf size, and max_features. LightGBM uses the tuned parameter set reported above for the current top-feature run; this run did not rerun random hyperparameter search. Its threshold search uses OOF probabilities over the grid $[0.44, 0.54]$ with step 0.005 and selects the threshold that maximizes CV accuracy. CatBoost varies iteration budget, depth, learning rate, and l2_leaf_reg. This intentionally small search design reflects recent evidence that strong pre-tuned defaults can already be competitive on tabular benchmarks, especially when the feature representation is stable and the evaluation loop is well controlled [5], [22].

TABLE V
CURRENT TUNING OUTCOMES RECORDED IN THE REPOSITORY

Model	Search setup	Best CV score	Best parameters
Logistic Regression	3-fold random search, 5 trials	0.7866	C=0.01; max_iter=1000; penalty=l2; solver=lbfgs.
Random Forest	3-fold random search, 5 trials	0.8069	max_depth=14; max_features=sqrt; min_samples_leaf=2; min_samples_split=10; n_estimators=200.
XGBoost	Not included in this lightweight tuning snapshot	0.8223	Maintained separately through the dedicated XGBoost validation and submission workflow.
LightGBM	5-fold top-feature refit, 1 selected run	0.8177	learning_rate=0.03; num_leaves=23; min_child_samples=40; subsample=0.85; colsample_bytree=0.75; reg_alpha=0.2; reg_lambda=2.5; threshold=0.500; final_n_estimators=350; selected_features=30.
CatBoost	3-fold random search, 5 trials	0.8138	depth=6; iterations=800; l2_leaf_reg=1; learning_rate=0.05.

The dash in the XGBoost row refers only to this lightweight cross-model tuning snapshot. It does not mean that the XGBoost branch was absent or untuned; that branch was maintained separately with a dedicated StratifiedGroupKFold, fold-aware preprocessing, Optuna-based tuning, ablation, leak diagnosis, and submission-selection workflow. Therefore, Table V should be read as the shared lightweight tuning summary, while the XGBoost-specific evidence is reported as a separate final-submission branch.

VI. RESULTS

A. Baseline Validation Results

Table VI summarizes the baseline validation run validation_001. The selected LightGBM top-feature run is reported separately because it uses 5-fold OOF CV and feature-ablation refitting rather than the single holdout validation layer.

The baseline result table already provides the full numeric ranking. The standalone accuracy bar chart is moved to the appendix to keep the main paper layout compact. Under this local validation protocol, XGBoost ranks behind LightGBM and CatBoost by accuracy, but remains competitive in ROC-

TABLE VI
BASELINE LOCAL VALIDATION RESULTS

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Fit (s)	Predict (s)
LightGBM	0.8171	0.8178	0.8196	0.8187	0.9095	1.6837	0.0383
CatBoost	0.8154	0.8129	0.8231	0.8179	0.9125	44.4018	0.0398
XGBoost	0.8114	0.8079	0.8208	0.8143	0.9119	1.0165	0.0221
Random Forest	0.8091	0.8269	0.7854	0.8056	0.9038	0.8591	0.2625
Logistic Regression	0.7907	0.7783	0.8174	0.7973	0.8790	0.5503	0.0132

TABLE VII
SELECTED LIGHTGBM TOP-FEATURE RUN

Item	Value
Model	LightGBM top-feature
Selected features	30
CV accuracy at threshold 0.5	0.8177
Tuned CV accuracy	0.8177
Best threshold	0.500
Final trees	350

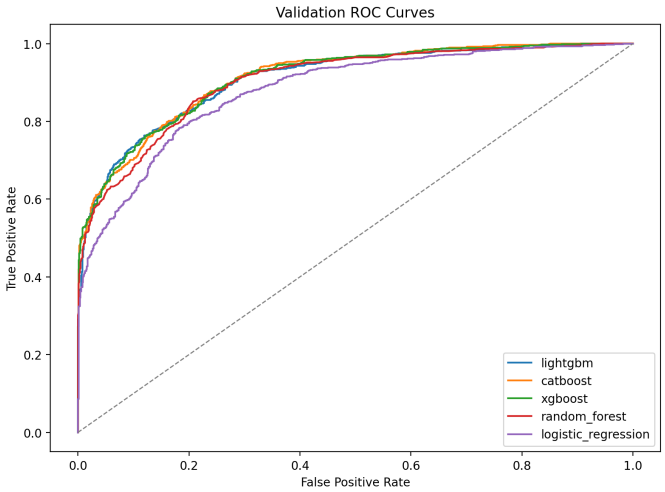


Fig. 6. ROC curves across all models.

AUC and runtime. This statement refers only to the local validation layer; the later Kaggle public result should not be used to reorder the local validation ranking.

B. Overall Model Comparison

Table VIII summarizes the training accuracy, local validation accuracy, local validation ROC-AUC, and retained Kaggle public leaderboard evidence for the five evaluated models. The table should be interpreted along two separate axes: local validation performance and external public leaderboard feedback.

The results show that the strongest model depends on the evaluation dimension. LightGBM obtains the highest local validation accuracy, reaching 0.8171, which indicates the best thresholded classification performance on the local holdout split. CatBoost obtains the highest validation ROC-AUC, reaching 0.9125, which suggests that its probability ranking quality is slightly stronger than the other models under the same local validation setting. However, the strongest retained Kaggle public leaderboard evidence comes from the XGBoost branch, with a retained public score of 0.81412. The package also retains the analyzed LightGBM top-feature branch public score of 0.80547. Other historical CatBoost, Random Forest, and Logistic Regression public-score records are not used as package evidence because their original leaderboard screenshots or submission records were not preserved in the final evidence folder.

This distinction is important because local validation metrics and Kaggle public leaderboard scores measure different forms of evidence. The local validation scores are produced under the controlled holdout protocol, whereas the public leaderboard scores reflect external feedback from submitted prediction files. Therefore, LightGBM can be described as the local validation accuracy leader, CatBoost as the local ROC-AUC leader, and XGBoost as the strongest retained public leaderboard submission in the current workspace.

The training accuracy column also helps diagnose overfitting. Random Forest reaches 1.0000 training accuracy but only 0.8091 validation accuracy, indicating clear overfitting despite competitive validation performance. LightGBM also shows a large train-validation gap, although it still achieves the best local validation accuracy. In contrast, Logistic Regression has much closer train and validation accuracy, but its lower ROC-AUC indicates limited model capacity for this feature representation.

C. Threshold-independent and Threshold-dependent Behavior

The current project already stores per-sample validation probabilities, which makes threshold-independent analysis possible through ROC curves and threshold-dependent analysis possible through threshold sweeps. The large multi-panel threshold figures are moved to the appendix so they do not break the main paper layout.

D. Error and Runtime Analysis

The project also includes side-by-side confusion matrices and runtime plots. The multi-panel confusion-matrix figure and the standalone runtime comparison chart are moved to the appendix so that the main text keeps a more compact narrative flow.

E. Optimization Impact

Lightweight tuning does not uniformly improve all models. This is a meaningful experimental result in itself: for the current feature branches, branch design and leakage-aware preprocessing appear to matter at least as much as shallow random search. Additional package figures, including calibration curves, precision-recall curves, probability histograms, runtime trade-off plots, boosting-iteration proxies, and tuned-versus-baseline charts, are retained in the report folder but omitted from the main paper body to keep float pressure manageable. This interpretation is consistent with recent data-centric benchmark work showing that preprocessing and representation choices can dominate marginal hyperparameter gains on tabular tasks [5], [6].

TABLE VIII
OVERALL COMPARISON OF LOCAL VALIDATION PERFORMANCE AND RETAINED KAGGLE PUBLIC LEADERBOARD EVIDENCE

Model	Train Accuracy	Validation Accuracy	Validation ROC-AUC	Retained Public LB Evidence
XGBoost	0.8834	0.8114	0.9119	0.81412
CatBoost	0.8622	0.8154	0.9125	Not retained
LightGBM	0.9617	0.8171	0.9095	0.80547
Random Forest	1.0000	0.8091	0.9038	Not retained
Logistic Regression	0.7916	0.7907	0.8790	Not retained

F. Feature Interpretation

The project provides both per-model importance plots and a normalized cross-model heatmap. This is important because raw importance values are not directly comparable across linear, bagged-tree, and boosting models, and because different importance definitions can lead to different selected subsets or narratives [19], [27], [28]. The normalized heatmap is retained in the appendix together with other supplementary diagnostics.

VII. KAGGLE SUBMISSION TABLE

The Kaggle table reports retained public leaderboard evidence for the part covered here. Public scores are listed only when the package preserves a corresponding screenshot, submission file, metadata record, or branch-level evidence. These public scores are external submission feedback and are not interpreted as private-leaderboard guarantees. Keeping leaderboard evidence separate from controlled local validation follows current guidance on more reproducible machine-learning reporting [8].

TABLE IX
KAGGLE PUBLIC LEADERBOARD EVIDENCE FOR SELECTED SUBMISSIONS

Model	Public LB	Notes
XGBoost	0.81412	Private LB not claimed; documented final XGBoost public submission.
LightGBM	0.80547	Private LB not claimed; best recorded public score within the analyzed top-feature LightGBM branch.

Historical public-score notes without retained package evidence are not used in this table. Therefore, CatBoost, Random Forest, and Logistic Regression are compared through the controlled local validation columns in Table VIII, while the retained public-score evidence is limited to XGBoost and the analyzed LightGBM top-feature branch.

Figure 7 records the external Kaggle feedback for the best team submission used in this report. **Code Availability.** The project code is available at https://github.com/u430034056-tech/epoch-MLW.git. The submitted ZIP package also includes a source-code snapshot and local reproduction scripts for offline inspection.

Across all recorded LightGBM submissions in the repository, the highest public leaderboard score is 0.80710 from submission_lightgbm_no_rule.csv; however, the present report focuses on the separately analyzed top-feature LightGBM branch, whose best recorded public score is 0.80547.

Fig. 7. Kaggle public leaderboard feedback for the best recorded team submission. The screenshot shows the submitted file submission_team_common_rawlike_feedback_rank_k3_seed17_w05_true2291.csv with a public score of 0.81412. This evidence is used only as public leaderboard feedback and is not interpreted as a private-leaderboard guarantee.

VIII. DISCUSSION

Several implementation-driven observations are worth emphasizing:

- The preprocessing layer is not generic boilerplate; it encodes domain-motivated rules such as CryoSleep zero-spend enforcement, group-consistent category completion, and train-only surname frequency mapping, which is closer to a data-centric tabular workflow than to a purely model-centric one [6], [7], [9].
- LightGBM benefits from a compact branch with native categorical handling; the selected 30-feature gain-ranked run achieves a tuned 5-fold CV accuracy of 0.8177 and the best recorded public leaderboard score of 0.80547 within the analyzed top-feature LightGBM branch, consistent with recent evidence that strong tree-based defaults remain hard to beat on typical tabular tasks [3], [5].
- CatBoost benefits from retaining Surname and other high-cardinality categorical signals directly, but this comes with much higher training time [13], [14].
- XGBoost is not the strongest local-validation model by accuracy, but it remains competitive in ROC-AUC and runtime. Its main contribution is a separate validated submission branch with group-aware validation, fold-aware feature recomputation, Optuna tuning, leak diagnosis, SHAP/importance evidence, multi-seed bagging, and a documented final public submission score of 0.81412.
- Logistic Regression is the weakest model by raw score but remains a useful linear baseline; its interpretability should still be understood as relative rather than automatic [17].

IX. LIMITATIONS AND FUTURE WORK

This study has several limitations:

- The local model comparison currently relies on a single stratified holdout rather than repeated cross-validation.
- The full preprocessing layer uses combined train+test group size for final bundle generation, which is accept-

able for inference-oriented preprocessing but must remain split-local during validation.
- XGBoost is not included in the shared lightweight tuning snapshot; its dedicated Optuna tuning, ablation, and final submission branch are therefore reported separately from Table V.
- Full per-iteration convergence histories are not persisted by the validation runner.
- The reported XGBoost and LightGBM public leaderboard scores are external submission feedback and should not be treated as private-leaderboard guarantees.
- The final XGBoost file was selected using documented Kaggle public feedback; this should be described as final submission selection, not as proof that a public-feedback perturbation is a generally superior modeling method.

Immediate next steps for the XGBoost branch include repeated cross-validation, broader XGBoost tuning under the same validation protocol, and additional Kaggle submissions only if new XGBoost candidates are actually submitted. More broadly, future work could evaluate stronger missing-data pipelines such as automatic model-selection-based imputation, compare against modern deep tabular baselines, and place the current workflow in a larger repository-style benchmark context [8], [12], [22], [29].

X. Contribution Statement

All five team members contributed equally to the project. Each member's contribution is assigned as 20% of the total work. The detailed division of work is summarized in Table X.

TABLE X
Team contribution statement

Member	Main responsibilities	Share
Wang Han	Logistic Regression feature engineering and model construction; report writing and integration; PPT production.	20%
Shen Yijie	Shared data preprocessing; XGBoost feature engineering and model construction; report writing and integration; PPT production; overall project integration and local reproduction.	20%
Yang Yusong	Exploratory data analysis; LightGBM feature engineering and model construction; report writing and integration; PPT production.	20%
Jiang Zhibo	Exploratory data analysis; Random Forest feature engineering and model construction; report writing and integration; PPT production.	20%
He Yuxiang	CatBoost preprocessing, feature engineering, and model construction; report writing and integration; PPT production and integration.	20%

XI. Conclusion

This study developed a leakage-aware preprocessing and validation workflow for the Spaceship Titanic classification task and compared five supervised tabular models under a common evaluation protocol. The local evidence is metric-dependent: LightGBM leads local validation accuracy, CatBoost leads ROC-AUC, and XGBoost is the final branch with the strongest recorded public leaderboard submission. For final external submission feedback, the selected XGBoost branch reached a Kaggle public leaderboard score of 0.81412. The main remaining limitation is that the cross-model comparison still relies on a single stratified holdout split rather than repeated cross-validation, which should be addressed in future work.

Appendix A
Supplementary Figures

Large diagnostic figures are collected here so the main body stays readable while the supporting plots remain available for inspection.

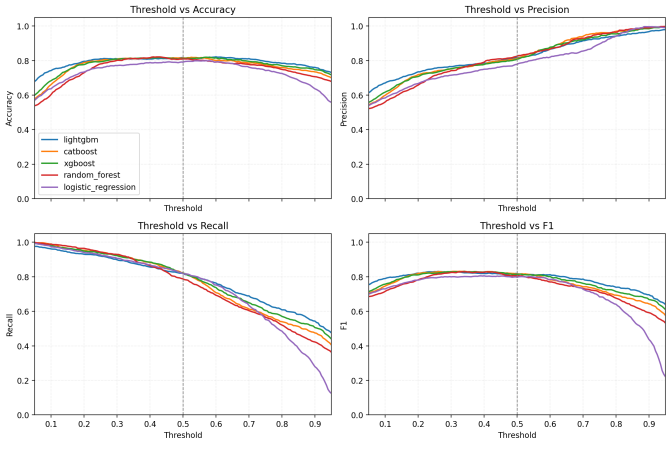


Fig. 8. Global threshold overlays for accuracy, precision, recall, and F1 across the five evaluated models on the local validation split. Accuracy and F1 remain relatively stable in the mid-threshold region, while precision increases and recall decreases as the decision threshold becomes more conservative. The dashed vertical line at 0.5 marks the default threshold and shows that it provides a reasonable overall trade-off without model-specific threshold tuning.

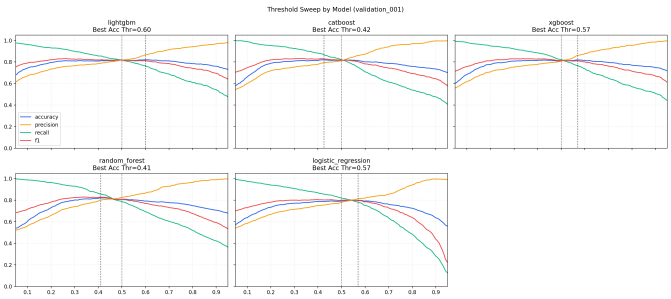


Fig. 9. Model-specific threshold sweeps for the local validation split. Each panel shows how accuracy, precision, recall, and F1 change with the classification threshold, while the dotted line marks the accuracy-maximizing threshold for that model. LightGBM and XGBoost achieve their best plotted accuracy at slightly higher thresholds, whereas CatBoost and Random Forest peak at lower thresholds, indicating that the default threshold of 0.5 is reasonable but not always optimal for every classifier.

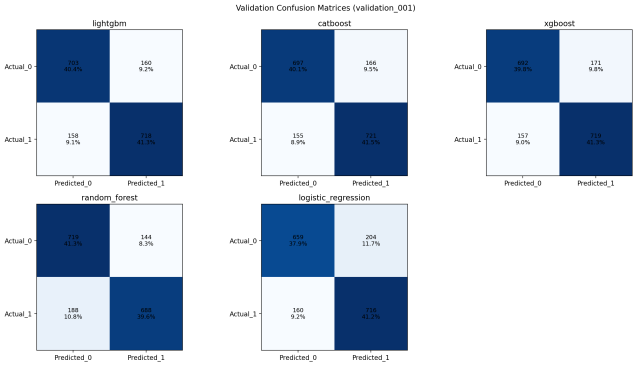


Fig. 10. Confusion matrices for the local validation split. LightGBM, CatBoost, and XGBoost show the most balanced error profiles, with strong true-negative and true-positive counts and comparatively limited off-diagonal errors. Random Forest is slightly more conservative and misses more positive cases, while Logistic Regression produces the largest false-positive count, which is consistent with its lower overall precision and accuracy.

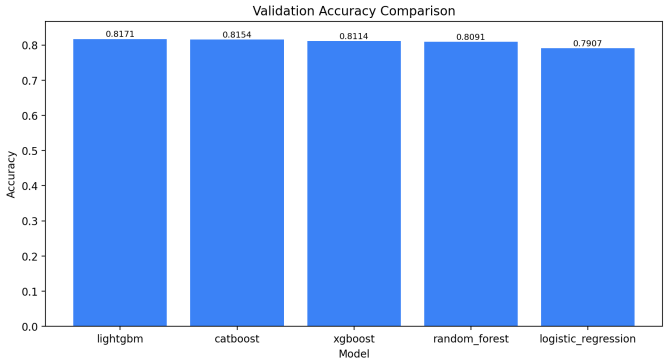


Fig. 11. Accuracy ranking across the five evaluated models on the local validation split. LightGBM achieves the highest validation accuracy (0.8171), followed closely by CatBoost (0.8154) and XGBoost (0.8114), while Random Forest remains competitive and Logistic Regression ranks last. The small gap among the top tree-based models suggests that the leading methods perform similarly under the same preprocessing and evaluation protocol.

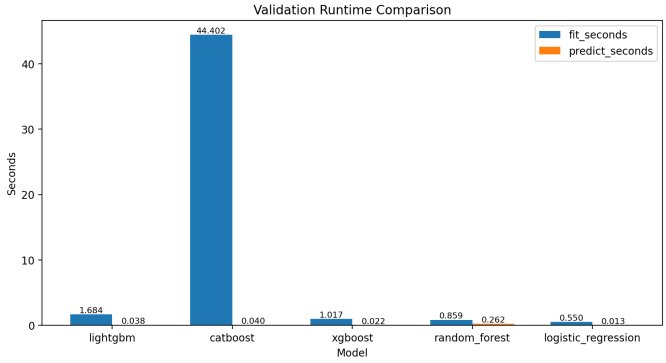


Fig. 12. Training and inference runtime comparison across models. Logistic Regression, XGBoost, and LightGBM train quickly, whereas CatBoost has by far the largest training cost at about 44.4 seconds. Prediction time remains low for all models, although Random Forest shows relatively slower inference, so the main efficiency difference in this experiment comes from training rather than deployment latency.

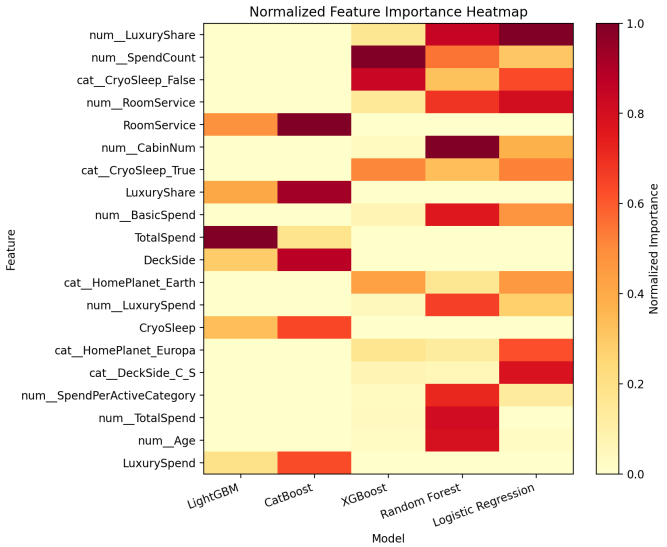


Fig. 13. Normalized feature-importance heatmap across all models. Spending-related variables, CryoSleep indicators, and cabin or location features appear repeatedly among the most influential predictors, but the relative emphasis differs by algorithm. Because all values are normalized before plotting, this figure should be interpreted as a comparison of importance patterns across models rather than a comparison of absolute importance magnitudes.

References

[1] Kaggle, "Spaceship Titanic," 2022. [Online]. Available: https://www.kaggle.com/competitions/spaceship-titanic

[2] V. Borisov et al., "Deep neural networks and tabular data: A survey," IEEE TNNLS, vol. 35, no. 6, pp. 7499–7519, 2024.

[3] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" NeurIPS, 2022.

[4] D. C. McElfresh et al., "When do neural nets outperform boosted trees on tabular data?" NeurIPS, 2023.

[5] D. Holzmuller, L. Grinsztajn, and I. Steinwart, "Better by default: Strong pre-tuned MLPs and boosted trees on tabular data," NeurIPS, 2024.

[6] A. Tschalzev et al., "A data-centric perspective on evaluating machine learning models for tabular data," NeurIPS, 2024.

[7] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, 2023.

[8] S. Kapoor et al., "REFORMS: Consensus-based recommendations for machine-learning-based science," Science Advances, vol. 10, no. 18, 2024.

[9] D. Chicco, L. Oneto, and E. Tavazzi, "Eleven quick tips for data cleaning and feature engineering," PLOS Computational Biology, vol. 18, no. 12, 2022.

[10] J. Josse et al., "On the consistency of supervised learning with missing values," Statistical Papers, vol. 65, no. 9, pp. 5447–5479, 2024.

[11] Y. Zhou, S. Aryal, and M. R. Bouadjenek, "Review for handling missing data with special missing mechanism," arXiv preprint arXiv:2404.04905, 2024.

[12] D. Jarrett et al., "Hyperimpute: Generalized iterative imputation with automatic model selection," arXiv:2206.07769, 2022.

[13] F. Matteucci, V. Arzamasov, and K. Bohm, "A benchmark of categorical encoders for binary classification," NeurIPS, 2023.

[14] B. Avanzi, G. Taylor, M. Wang, and B. Wong, "Machine learning with high-cardinality categorical features in actuarial applications," arXiv:2301.12710, 2023.

[15] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," JMLR, vol. 12, pp. 2825–2830, 2011.

[16] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," Nature Machine Intelligence, vol. 1, no. 5, pp. 206–215, 2019.

[17] D. Dervovic et al., "Are logistic models really interpretable?" IJCAI, 2024.

[18] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.

[19] G. Louppe, L. Wehenkel, A. Sutera, and P. Geurts, "Understanding variable importances in forests of randomized trees," in Proc. 26th Int. Conf. Neural Information Processing Systems (NIPS), 2013, pp. 431–439.

[20] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," KDD, pp. 785–794, 2016.

[21] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," NeurIPS, vol. 30, 2017.

[22] D. Salinas and N. Erickson, "TabRepo: A large scale repository of tabular model evaluations and its AutoML applications," ICML AutoML, 2024.

[23] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in Proc. 32nd Int. Conf. Neural Information Processing Systems (NeurIPS), 2018, pp. 6639–6649.

[24] T. Silva Filho et al., "Classifier calibration: A survey on how to assess and improve predicted class probabilities," Machine Learning, vol. 112, no. 9, 2023.

[25] D. Lee, X. Huang, H. Hassani, and E. Dobriban, "T-cal: An optimal test for the calibration of predictive models," JMLR, vol. 24, no. 335, 2023.

[26] J. L. Leevy et al., "Threshold optimization and random undersampling for imbalanced credit card data," Journal of Big Data, vol. 10, no. 1, 2023.

[27] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," NeurIPS, vol. 30, 2017.

[28] C. Strobl, A.-L. Boulesteix, A. Zeileis, and T. Hothorn, "Bias in random forest variable importance measures: Illustrations, sources and a solution," BMC Bioinformatics, vol. 8, no. 25, 2007.

[29] S. Somvanshi et al., "A survey on deep tabular learning," arXiv:2410.12034, 2024.